

STACK: Intuition

A **Stack** is a **linear data structure** that follows the **Last In, First Out (LIFO)** principle:

- **Think of a stack of plates:** you add plates on top and remove plates from the top.

Key Operations

Operation	Description
<code>push(x)</code>	Add element <code>x</code> to the top of the stack
<code>pop()</code>	Remove and return the top element
<code>peek()</code> / <code>top()</code>	View the top element without removing it
<code>isEmpty()</code>	Check if the stack is empty
<code>size()</code>	Return number of elements

Applications of Stack

1. **Function call management** (Call stack)
2. **Undo/Redo operations**
3. **Expression evaluation** (`infix` $\rightarrow$ `postfix`)
4. **Balanced parentheses check**
5. **Backtracking algorithms** (maze, sudoku)
6. **Web browser history**
7. **Language parsing and compilation**

Implementation

Python (Using list)

```
class Stack:
    def __init__(self):
        self.stack = []
```

```

def push(self, val):
    self.stack.append(val)

def pop(self):
    if self.is_empty():
        return None
    return self.stack.pop()

def top(self):
    if self.is_empty():
        return None
    return self.stack[-1]

def is_empty(self):
    return len(self.stack) == 0

def size(self):
    return len(self.stack)

```

Java

```

import java.util.*;

class StackDemo {
    Stack<Integer> stack = new Stack<>();

    void push(int val) {
        stack.push(val);
    }

    int pop() {
        return stack.pop();
    }

    int top() {
        return stack.peek();
    }

    boolean isEmpty() {
        return stack.isEmpty();
    }

    int size() {
        return stack.size();
    }
}

```

C++

```
#include <iostream>
#include <stack>
using namespace std;

class StackDemo {
    stack<int> s;

public:
    void push(int val) {
        s.push(val);
    }

    void pop() {
        if (!s.empty()) s.pop();
    }

    int top() {
        return s.top();
    }

    bool isEmpty() {
        return s.empty();
    }

    int size() {
        return s.size();
    }
};
```